

מה שנשמר נשאר

שבוע: 10 ניקוד: 100 מועד הגשה: לפני תחילת שיעור 11 משקל בציר הסופי: מטלת כיתה · 10 הטובות מתוך 12 נספרות זמן משוער: 90 דקות

מטרה

להוסיף ליישום שכבה רביעית שמוציאה את האוסף מהזיכרון אל הדיסק — בלי לגעת בשלוש הקיימות — ולסגור את מעגל ה-CRUD באות שחסרה בו: **עריכה**.

בסוף השיעור הקודם הבטחתי משפט אחד: שהתמדה תהיה **שכבה חדשה ולא כתיבה מחדש**. היום בודקים את זה. `js/render.js` לא ילמד ששומרים משהו, `js/events.js` לא ילמד, `js/state.js` לא ילמד — ו- `js/app.js` יגדל בשורה אחת. ובסוף השיעור, לראשונה בסמסטר, תסגור את הלשונית ותפתח אותה מחר בבוקר והכול יהיה שם.

תוצרי למידה

- לדעת מה JSON יכול לייצג ומה **לא**, ומה `JSON.stringify` משמיט בשקט.
- לכתוב שכבת אחסון שבה **כל גישה** ל- `localStorage` עטופה, ולהסביר למה גם הקריאה.
- להבדיל בין "לא נשמר כלום" לבין "נשמר אוסף ריק" — שתי תשובות שונות לגמרי.
- להגן על היישום מפני מחרוזת פגומה, ומפני **JSON תקין לחלוטין שאינו הנתונים שלך**.
- לתת למפתח מרחב שמות וגרסה, ולהגר נתונים שנכתבו על ידי גרסה קודמת.
- לסגור מעגל CRUD מלא — הוספה, עריכה, מחיקה — שכל שלב בו שורד רענון.

מה יש בתיקייה

קובץ	מה עושים איתו
<code>js/storage.js</code>	הקובץ של היום . שלד עם חוזה, בלי מימוש
<code>js/app.js</code>	נתון. שתי שורות בו משתנות, והן מסומנות
<code>js/state.js</code>	נתון. שני שדות חדשים ופונקציה אחת
<code>js/render.js</code>	נתון. שלושה סימוני <code>CODE HERE</code> , אף אחד מהם לא אחסון
<code>js/events.js</code>	נתון. עריכה, ביטול, ניקוי הכול
<code>js/items.js</code>	הנתונים. מהיום הם רק נתוני הדוגמה
<code>summary.html</code> · <code>item.html</code> · <code>index.html</code>	נתונים ומלאים. לא נוגעים בהם

מריצים עם Live Server, תמיד. `localStorage` משויך ל-`origin`, ול-`file://` אין `origin` אמיתי – הנתונים שלך שם עלולים להיות משותפים לכל קובץ במחשב או לא להישמר בכלל.

המילה `localStorage` מופיעה בקובץ אחד בלבד. לא ב-`render.js`, לא ב-`events.js`, לא ב-`state.js`. אותו דבר לגבי `JSON`. חפש אותן בשלושת הקבצים כשסיימת.

אף מטפל אירועים לא קורא לשמירה בעצמו. ההרשמה ב-`js/app.js` היא כל החיווט. מטפל שמסתיים ב-`save()` עובד היום, ובעוד חודש מישהו יוסיף מטפל חמישי וישכח.

`js/storage.js` לא נוגע ב-DOM. אין בו `document` ואין בו `querySelector`, בדיוק כמו `js/state.js`.

כל גישה ל-`localStorage` בתוך `try` – לא רק ה-`parse`. `getItem` עצמו זורק בדפדפן שחוסם אחסון לאתר הזה, וזריקה בנתיב האתחול היא עמוד לבן.

מפתח אחד, עם מרחב שמות וגרסה. לא `items`, לא `data`.

`localStorage.clear()` אסור. הוא מוחק את כל ה-`origin`, כלומר גם את כל הפרויקטים האחרים שלך על אותו פורט.

חלק א · שכבת האחסון · 30 נקודות ליבה

א.1 · המפתח · 4 נקודות ליבה

מפתח אחד, ובו שלושה חלקים: **מרחב שמות · שם · גרסה.**

הסיבה אינה סגנון. `localStorage` משויך ל-`origin` – סכימה, מארח ופורט – ולא לתיקיה. כל פרויקט שאתה מריץ עם Live Server מוגש מאותו `http://127.0.0.1:5500`, ולכן מפתח בשם `items` הוא מפתח ששלושה יישומים שלך כבר נלחמים עליו. פתח את DevTools בלשונית Application, בחר Local Storage, והסתכל מה כבר יושב שם.

תקדים מהקורס עצמו

הקוד שלי מ-2026 כבר עשה חצי מזה בשורה אחת: שם המפתח היה `users_cache_v1`. החצי שהיה חסר לו הוא מרחב השמות, וזה בדיוק החצי שנשבר כשמריצים שני פרויקטים.

א.2 · `load` · 10 נקודות ליבה

מחזירה את האוסף השמור, או `null` אם אין מה לקרוא. **היא לא זורקת לעולם** – היא רצה לפני הציור הראשון, וזריקה שם היא עמוד לבן שנראה בדיוק כמו `render` שבור.

ארבע דרכים לקבל `null`, והקורא לא מבדיל ביניהן: האחסון חסום, לא נשמר כלום, המחרוזת אינה JSON, או שה-JSON אינו המבנה שלך.

המצב	מה מחזירים	ולמה
<code>getItem</code> זרק	<code>null</code>	הדפדפן חוסם אחסון לאתר הזה. לא סיבה לא לצייר עמוד
<code>getItem</code> החזיר <code>null</code>	<code>null</code>	ביקור ראשון. זו התשובה הרגילה, לא תקלה
<code>JSON.parse</code> זרק	<code>null</code>	מחרוזת פגומה. אל תמחק אותה – זה לא הרגע להחליט למשתמש

המצב	מה מחזירים	ולמה
נשמר אוסף ריק	[]	המשתמש מחק הכול. זו תשובה, ולא היעדר תשובה
המבנה אינו שלך	null	JSON תקין אינו נתונים תקינים

השורה החמישית היא זו שנופלים בה. `load` שמדווח על אוסף ריק כ"אין כלום" נותן למי שמחק את הכול את נתוני הדוגמה בחזרה ברענן הבא – ואי אפשר למצוא את זה בבדיקה ידנית, כי צריך לנקות הכול **ואז** לרענן, בסדר הזה, ואף אחד לא מרענן מיד אחרי שניקה.

JSON.parse(null) לא זורק

`getItem` על מפתח שלא קיים מחזיר `null`, ו-`JSON.parse(null)` הופך אותו למחרוזת `"null"` ומפרסר בחזרה ל-`null`. אין שגיאה, אין אזהרה – והקריסה מגיעה ארבע שורות אחר כך, ב-`map`. שבתוך `render`. שם גם יילך לך חצי שיעור.

3.א. `save` ו-`persist` • 12 נקודות • ליבה

`save(state)` כותבת רק את האוסף. לא את החיפוש, לא את המיון, ולא את השורה שנערכת: סינון שנשמר הוא באג שנקרא בדיוק כמו "הנתונים שלי נעלמו".

`setItem` היא הקריאה היחידה שנכשלת גם בדפדפן בריא – כשעוברים את המכסה. אל תבלע את השגיאה. `catch {}` ריק נותן למשתמש יישום שנראה בדיוק כמו אחד ששומר ואינו שומר, והוא מגלה כשהוא סוגר את הלשונית.

`persist(state)` היא המנוי. `js/app.js` רושם אותה פעם אחת, וזה כל החיווט. היא שומרת, ורק אם התשובה השתנתה היא מכניסה את הבעיה ל-`state` כדי ש-`render` יוכל לומר עליה משפט.

למה "רק אם השתנתה"

`persist` קוראת ל-`setState`, ו-`setState` מודיעה לכל המנויים – כולל לה. השמירה על כך שהיא מעדכנת רק כשיש שינוי היא מה שגורם לזה להיעצר.

4.א. • החיווט • 4 נקודות • ליבה

`js/app.js` מקבל שורה אחת חדשה ושינוי אחד. הסדר קובע:

#	השורה	למה בסדר הזה
1	קריאה מהדיסק, ונפילה לזריעה	לפני שמישהו נרשם, אחרת זה כותב מיד בחזרה את מה שהרגע קרא
2	<code>subscribe(render)</code>	המסך עוקב אחרי כל שינוי
3	<code>subscribe(persist)</code>	וגם הדיסק. הסדר בין 2 ל-3 לא משנה – אף אחת מהן לא קוראת את השנייה
4	<code>wire()</code>	פעם אחת, ולא מתוך <code>render</code>
5	<code>render(getState())</code>	עדיין השורה ששוכחים

עד היום היו לך שלוש אותיות מתוך ארבע: יצירה, קריאה, מחיקה. **עריכה** היא זו שחסרה, והיא גם זו שהופכת התמדה למשהו ששווה משהו — פריט שאפשר רק ליצור ולמחוק לא באמת צריך לשרוד את הלילה.

1. **כפתור `button.edit` בכל שורה.** `render` בונה אותו, בדיוק כמו שני האחרים.
2. **לחיצה פותחת את הפריט בטופס הקיים.** אותו טופס. מה שמשתנה הוא מה שקורה בשליחה.
3. **שליחה מחליפה את הפריט,** ולא מוסיפה אחד חדש: אותו מזהה, אותה כמות שורות. `map` עם `spread`, ובלי לגעת בפריט המקורי.
4. **המזהה ותאריך היצירה לא ניתנים לעריכה** — הם מגיעים מה-`spread` של הפריט הישן ואינם בין השדות שהטופס מחזיר.
5. **`#form-cancel`** יוצא ממצב עריכה בלי לשנות כלום.
6. **מצב ריק.** כשאין פריטים יש משפט; כשיש, אין. **וגם הריקנות נשמרת** — מי שמחק הכול ורענן צריך לראות את אותו משפט, ולא את נתוני הדוגמה.

הטופס הוא היוצא מן הכלל היחיד, והוא כבר היה כזה

`render` רץ בכל הקשה בתיבת החיפוש. שדה שהערך שלו נדרס בזמן שמשהו מקליד בו הוא שדה שנלחם במשתמש — ולכן **שדות הטופס אינם מצוירים מ-`state`.** `events.js` ממלא אותם ברגע שהעריכה מתחילה וקורא אותם בשליחה. מה ש-`render` כן מצייר הוא **המצב** של הטופס: תווית הכפתור, כפתור הביטול, והסימון על השורה שנערכת. הכלל, ושווה לזכור אותו לפרויקט: **נתונים מצוירים מ-`state`; הערך של פקד — לא.**

בסוף השיעור פתח את DevTools, לשונית Application, ותערוך את הערך של המפתח שלך ביד. מחק סוגר. שים `null`. שים `[1, 2, 3]`. רענן אחרי כל אחד. **היישום חייב לעלות בכל אחד מהם,** ולהראות משהו — נתוני זריעה או מצב ריק, לא עמוד לבן ולא שגיאה בקונסול.

ארבעה מטענים, ורק על אחד מהם `try/catch` עוזר:

המטען	<code>JSON.parse</code>	מה שתופס אותו
<code>{"v":1,"ite</code>	זורק	<code>try/catch</code>
אין מפתח	לא מגיעים	בדיקת <code>null</code>
<code>null</code>	מצליח	בדיקת מבנה
<code>{"items":"hello"}</code>	מצליח	בדיקת מבנה

שלושה מתוך ארבעה עוברים את ה-`parse` בשלום. `try/catch` הוא רבע מהתשובה.

בדיקת המבנה בודקת את השדות שהיישום באמת **קורא**, עם הטיפוסים שהוא באמת מניח. לא יותר — ולידציה שמשכפלת את כל הסכמה היא הגדרה שנייה של הפריט, וזה בדיוק מה שהקורס הזה מנקה ממנו סמסטר שלם.

ג.1 · ניקוי הכול · 4 נקודות ליבה

`#clear-all` מוחק את כל הפריטים, ושואל קודם.

בשבוע 9 הכלל היה: העדף ביטול על פני אישור, ואשר רק את מה שאי אפשר לבטל. זה מה שאי אפשר לבטל.

המלכודת, והיא הבאג הכי יפה של השבוע

"לנקות את הנתונים של המשתמש" פירושו **לשמור אוסף ריק**. זה לא `removeItem`.

מפתח שאינו קיים אומר "הדפדפן הזה מעולם לא היה כאן", ויישום שזורע נתוני דוגמה בביקור ראשון ישמח לשים אותם בחזרה. כלומר: ארבעים פריטים אמיתיים נמחקו, ואחרי הרענון הבא שש דוגמאות על המסך. זה עובר כל בדיקה ידנית, כי אף אחד לא מרענן מיד אחרי שניקה.

`removeItem` מחובר למקום אחד ואחד בלבד — ההצעה שבתוך המצב הריק לשחזר את נתוני הדוגמה, כלומר הפקד היחיד שמשמעותו באמת "תעשה כאילו לא הייתי כאן".

חלק ד · גרסה ומיגרציה · 20 נקודות הרחבה

היישום עלה לאוויר, אנשים השתמשו בו, ואז המבנה של פריט השתנה. הדפדפנים שלהם עדיין מחזיקים את המבנה הישן, וזה העותק היחיד של הנתונים שלהם שיש.

1. מה שנשמר הוא מעטפה, לא מערך חשוף: אובייקט שיש בו האוסף ויש בו מספר גרסה.

2. `migrate` מחזירה שלוש תשובות ולא שתיים:

מה שנמצא	מה עושים
המעטפה הנוכחית	לוקחים את האוסף
מערך חשוף	מאמצים אותו. זה מה שהניסיון הראשון שלך הבוקר כתב, והנתונים שלמים לגמרי
כל השאר	<code>null</code> . גרסה שלא מוכרת עלולה להיות שונה בכל שדה, וניחוש משחית במקום לאבד

1. `migrate` היא טהורה: מקבלת את מה שנקרא, מחזירה אוסף. היא לא מדפיסה, לא מוחקת את המפתח ולא מציירת.

המילה השקטה

מיגרציה שמוחקת נתונים שהיא לא הבינה היא הדבר היחיד שגרוע ממיגרציה שאין. אם אתה לא יודע מה זה — אל תיגע בו, זרע מחדש, ותן למשתמש כפתור.

חלק ה · תקציב הכתיבות · 10 נקודות אתגר

`subscribe(persist)` היא שורה אחת והיא נכונה. היא גם כתיבה סינכרונית לדיסק **בכל שינוי state** — ואחרי שבוע 9, הקלדה בתיבת החיפוש היא שינוי `state`.

1. **מדוד**. עטוף את `setItem` במונה, הקלד מילה בתיבת החיפוש, וכתוב את המספר.

2. **תקן**, באחת משתי הדרכים:

• **שמירת זהות** – שמור את המערך האחרון שנכתב **לפי הפניה**, וצא מוקדם אם הוא אותו מערך. שורה אחת, והיא **מדויקת**, בזכות כלל משבוע 9 שאין לו שום קשר לאחסון: המטפלים בונים מערכים **חדשים** במקום לערוך את הקיים.

• **השהיה** – כתוב 300 מילישניות אחרי השינוי האחרון. הכלי הכללי, והוא נכון גם כשהאוסף עצמו משתנה מהר. **חובה flush** ב- **pagehide** : בלעדיו יש חלון שבו השינוי על המסך ואינו על הדיסק, וזה בדיוק החלון שבו סוגרים את הלשונית.

3. **כתוב את שני המספרים בהערה** ב- `js/storage.js`. השאלה היא כמותית.

כללי · 5 נקודות ליבה

מה	כמה
אין <code>var</code> · אין <code>innerHTML</code> עם טקסט שמשמש הקליד	2
היגיינת קומיטים: קומיט לכל חלק, הודעה שאומרת מה השתנה	3

מדיניות AI לשבוע הזה

שותף מימוש, כמו משבוע 7 – עם אזהרה שספציפית לשבוע הזה.

שכתב אחסון היא בדיוק סוג הקוד ש-Claude Code כותב מצוין: `setItem`, `JSON.parse`, `getItem`, `JSON.stringify`, וסיימנו. **וזו בדיוק הבעיה** – הגרסה שהוא כותב בברירת מחדל היא הגרסה שעובדת עד הפעם הראשונה שמשוה לא צפוי יושב במפתח.

שלוש טעויות שהוא עושה כאן שוב ושוב, ושלושתן עוברות כל בדיקה ידנית:

1. `try/catch` סביב `parse`-הבלבד, בלי בדיקת מבנה – ואז `{ "items": "hello" }` מפיל את היישום דווקא בגלל שהוא תקין.

2. `catch {}` ריק בכתיבה, שהופך כישלון מכסה לשקט מוחלט.

3. קריאה ל- `save()` בתוך כל מטפל, במקום מנוי אחד – כי ככה "בטוח יותר".

קרא את הפלט מול הטבלה של חלק ג לפני שאתה מריץ אותו, וגם מול השורה החמישית בטבלה של חלק א.2. שלוש הטעויות האלה עולות נקודות, ובפרויקט הן עולות נתונים.

הגשה

תיקיה `week-10` במאגר הסמסטר שלך.

הגשה במודל: כתובת המאגר + ה-SHA של הקומיט. ה-SHA מקפיא את מה שנבדק.

אחרי ההגשה, העתק את `js/storage.js` גם ל- `project/` שלך והוסף את שתי השורות ל- `js/app.js` שם. **הפרויקט מוגש בשבוע 12 עם נתונים שנשמרים**, וזה השבוע שבו זה נעשה – לא בשבוע 12.

הגשה מאוחרת: ההגשה נסגרת במועד שכתוב במודל. הגשה שמגיעה אחרי המועד מתקבלת ונבדקת במלואה, ומסומנת כמאוחרת – היא אינה מורידה נקודות.

- ☐ .
הוספת פריט, סגרת את הלשונית, פתחת מחדש – הוא שם.
- ☐ .
ערכת פריט, רעננת – **הערך החדש** שם, ולא הישן ולא שניהם.
- ☐ .
הסרת פריט, רעננת – הוא לא חזר.
- ☐ .
המילה `localStorage` לא מופיעה ב- `render.js`, ב- `events.js` וב- `state.js`.
- ☐ .
המילה `document` לא מופיעה ב- `storage.js`.
- ☐ .
פתחת Application ב-DevTools, השחתת את הערך ביד, ורעננת – העמוד עלה.
- ☐ .
שמת `null` בערך ורעננת – העמוד עלה.
- ☐ .
ניקית הכול, רעננת – עדיין ריק, ועם משפט.
- ☐ .
במפתח יש נקודה וגם `v` ומספר.
- ☐ .
רשמת בהערה כמה כתיבות עלתה הקלדת מילה לפני התיקון וכמה אחריו.